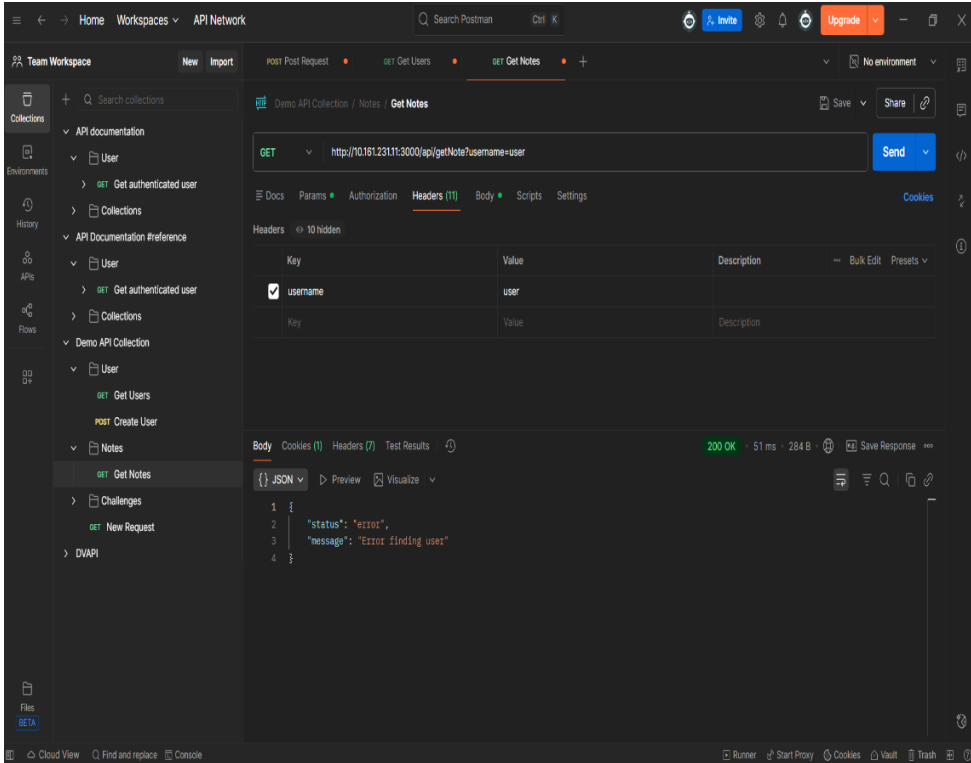
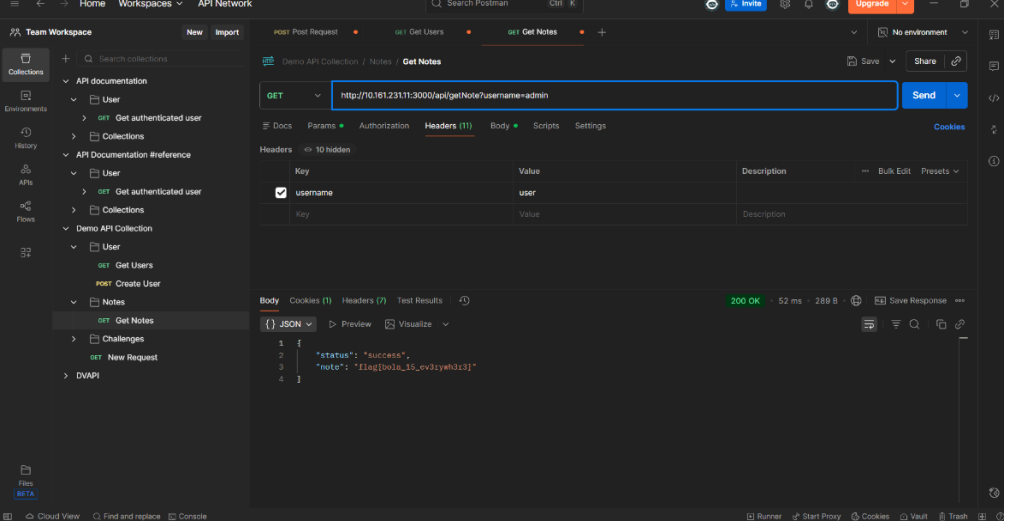
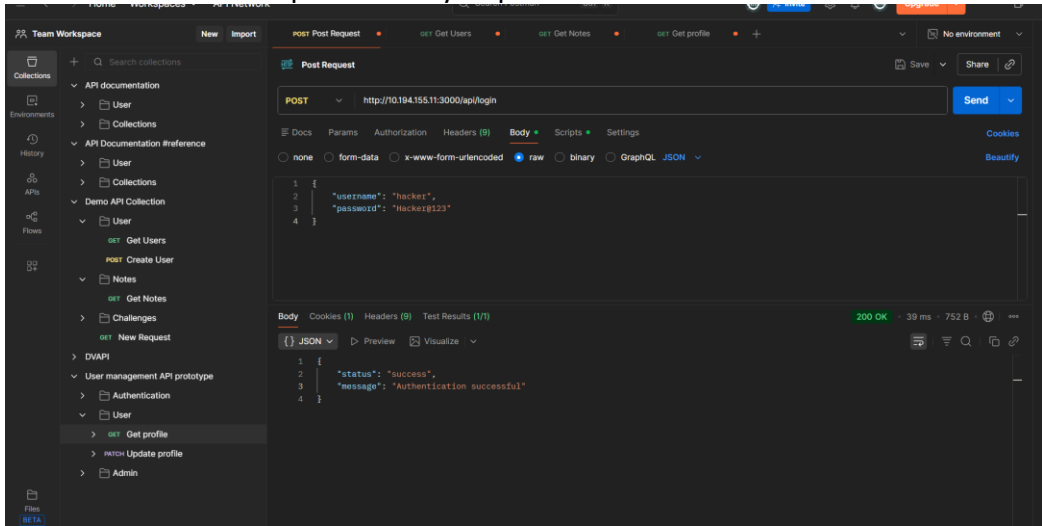
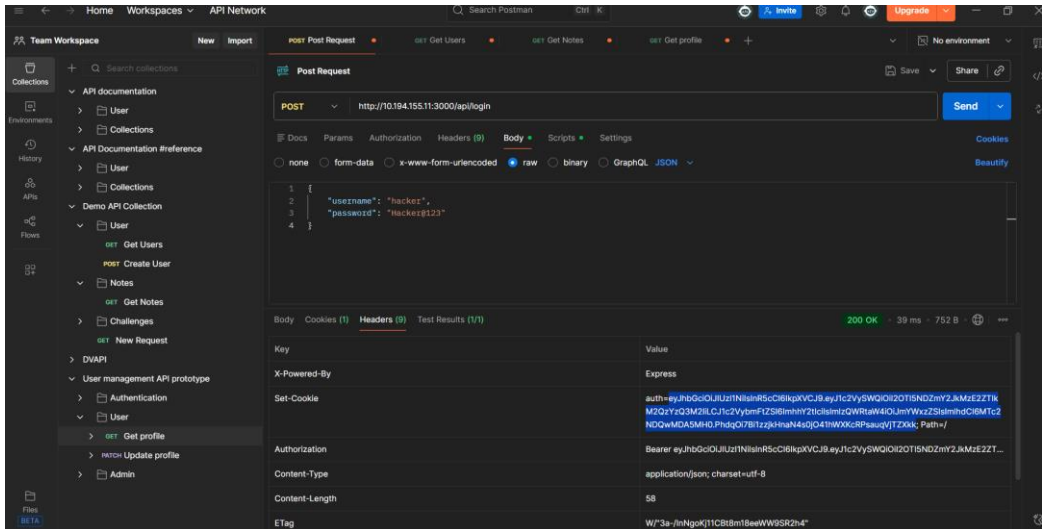
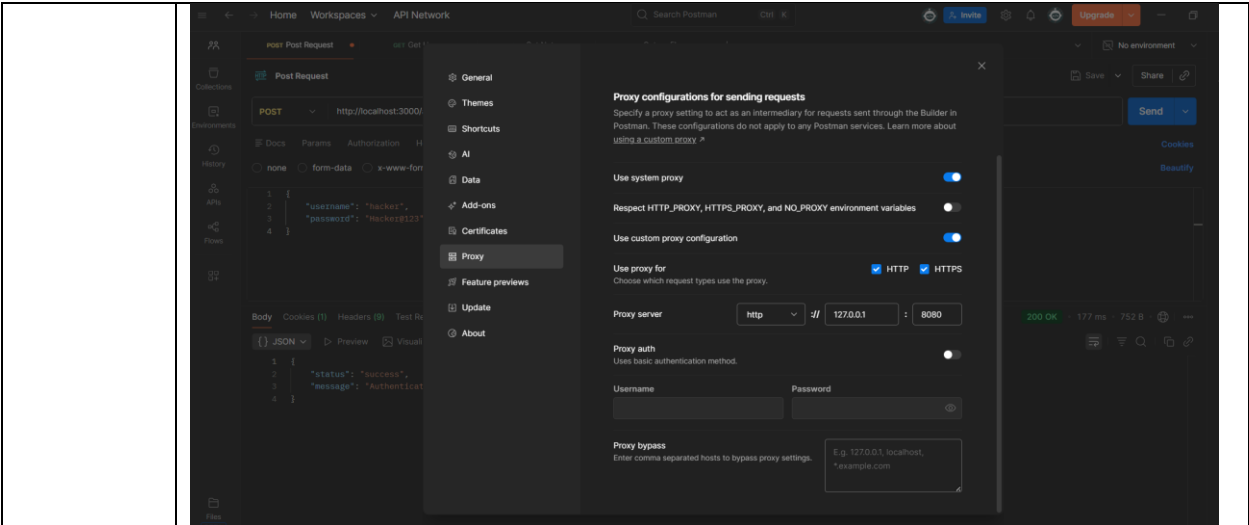


Vulnerability Name	Broken Object Level Authorization
Target URL	http://10.161.231.11:3000/api/getNote?username=
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – We have Endpoint where we can modify parameter and fetch the information about user  STEP 2 – Try to access other user data by passing different user value in parameter

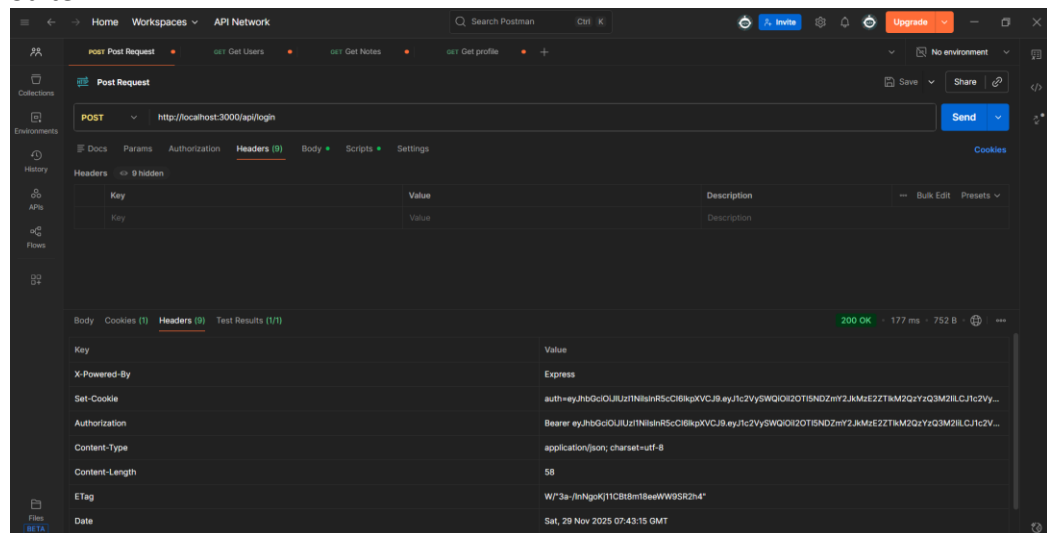
	
Impact	Unauthorized access to other users' objects can result in data disclosure to unauthorized parties, data loss, or data manipulation. Under certain circumstances, unauthorized access to objects can also lead to full account takeover
Remediation	• Implement a proper authorization mechanism that relies on the user policies and hierarchy. • Use the authorization mechanism to check if the logged in user has access to perform the requested action on the record in every function that uses an input from the client to access a record in the database. • Prefer the use of random and unpredictable values as GUIDs for record's IDs. • Write tests to evaluate the vulnerability of the authorization mechanism. Do not deploy changes that make the tests fail.
Reference	https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/

Vulnerability Name	Broken Authentication
Target URL	http://10.194.155.11:3000/api/login
Severity	Critical

CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – We have endpoint lets try to pass valid Credentials  STEP 2 – We have JWT Token here  STEP 3 – Let's forward this to Burp Suite by setting proxy in Postman

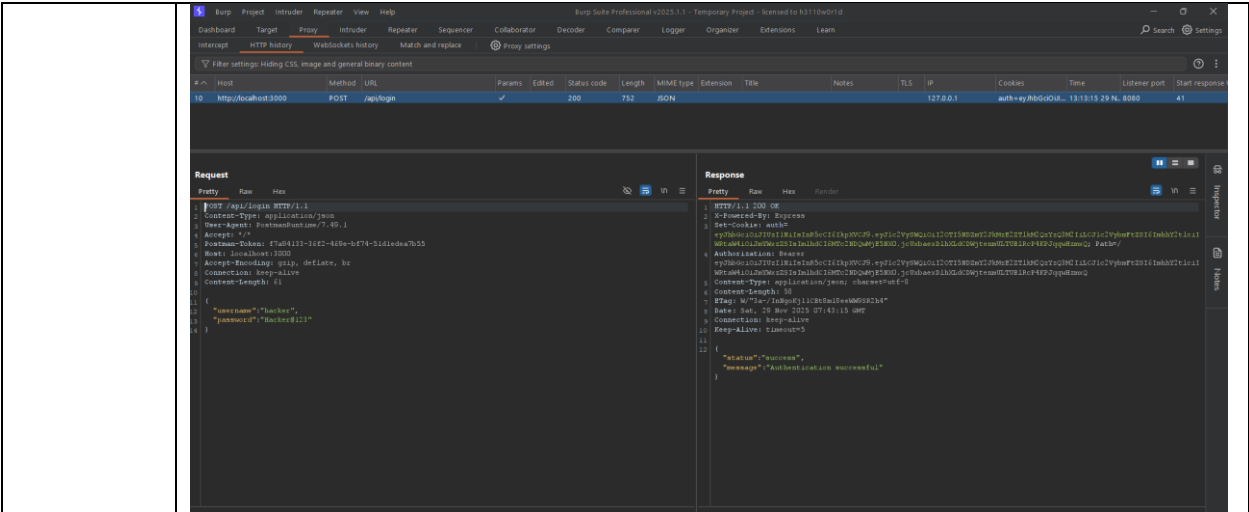


STEP 4 – Send the request from Postman and let's listen the request on Burp Suite

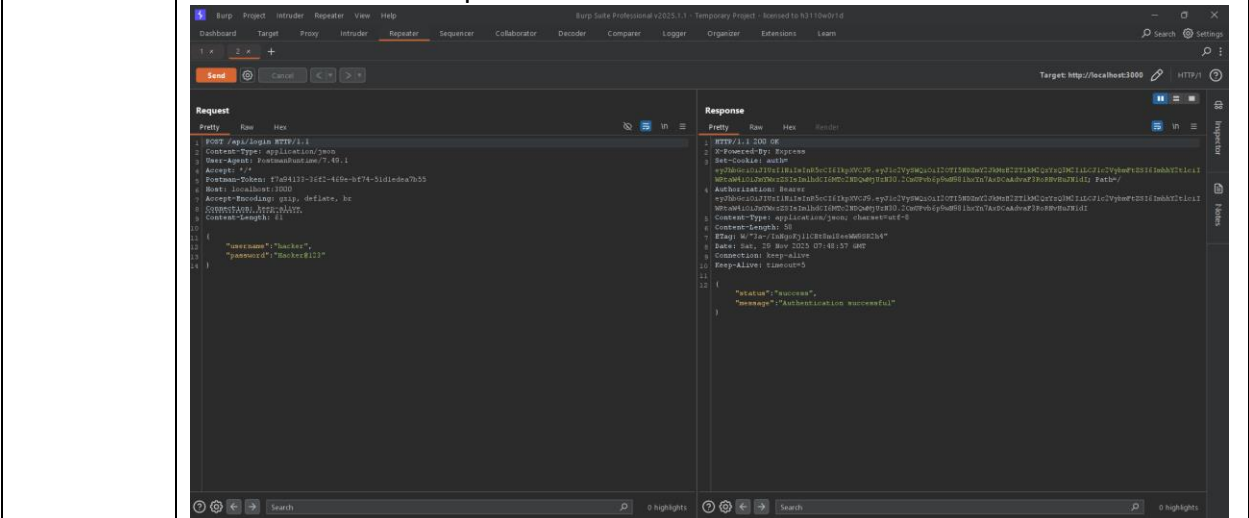


STEP 5 – We got Request on Burp Suite

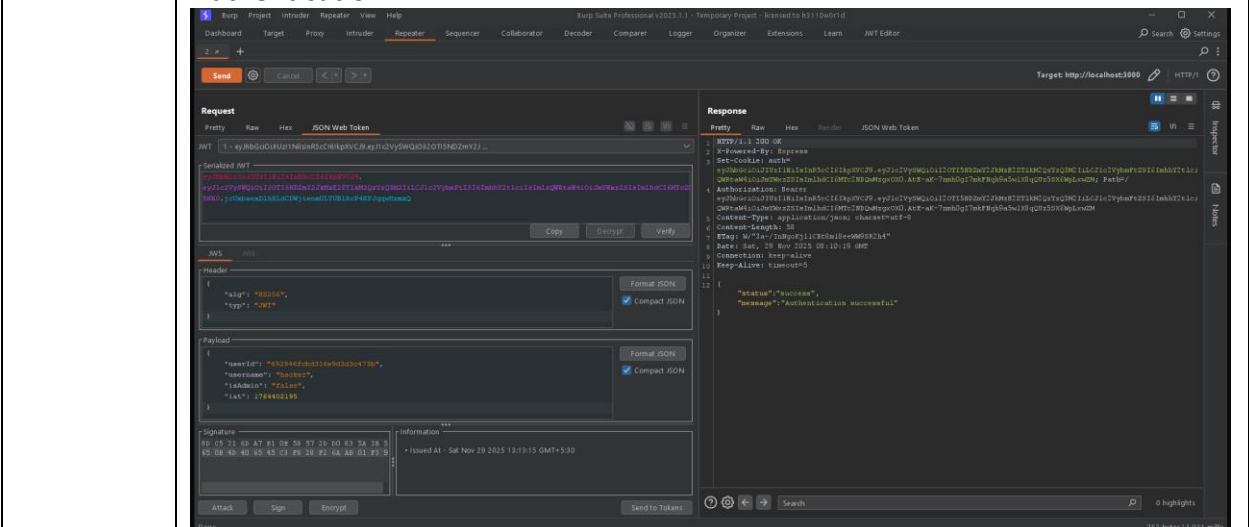
WEB PENETRATION TESTING - DVAPI



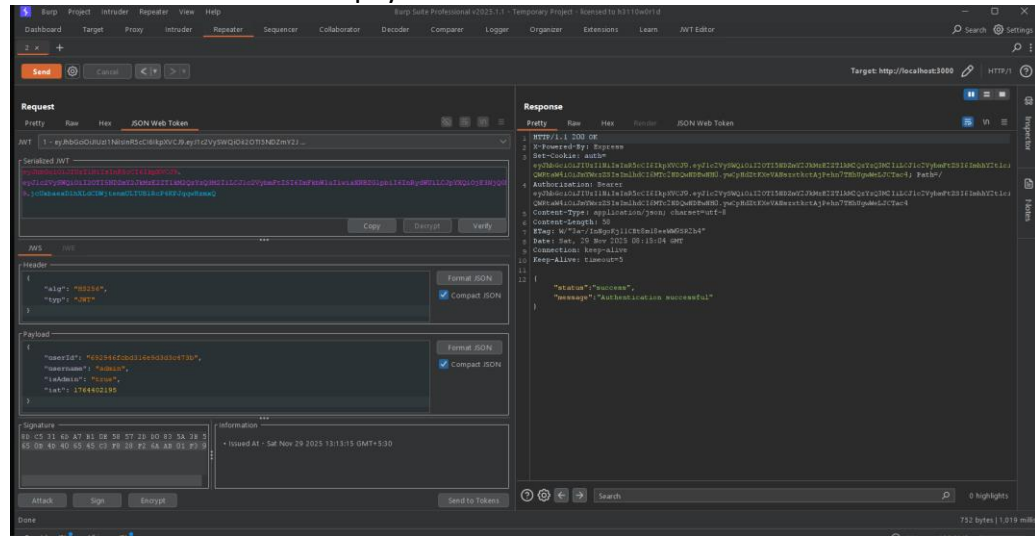
STEP 6 – Send it to repeater



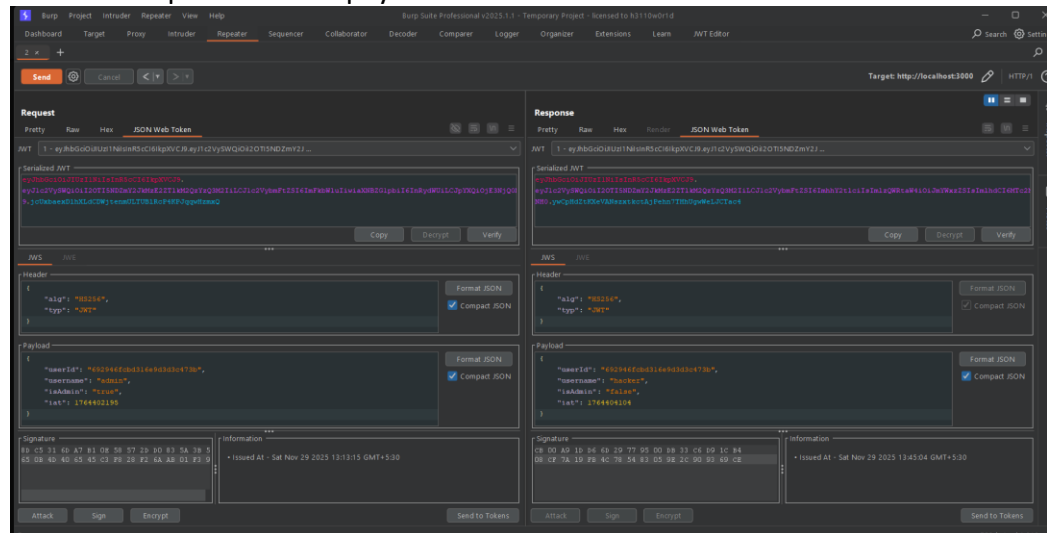
STEP 7 – Let's use JWT Editor and try to edit values in token and bypass Authentication



STEP 8 – In token we have payload which name is Admin with value true



- We got 200 OK successes but if we see token in response, we can see that parameter or payload value in token is still



- So, what's problem here is that our send token is not signed for admin it was signed for user hacker we need to sign token for admin
- To do so we need secrete key to sign the token

STEP 9 – Brute Force JWT Token to Find Secret

SYNTAX - hashcat jwt.txt -m 16500 /usr/share/wordlists/rockyou.txt

```

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

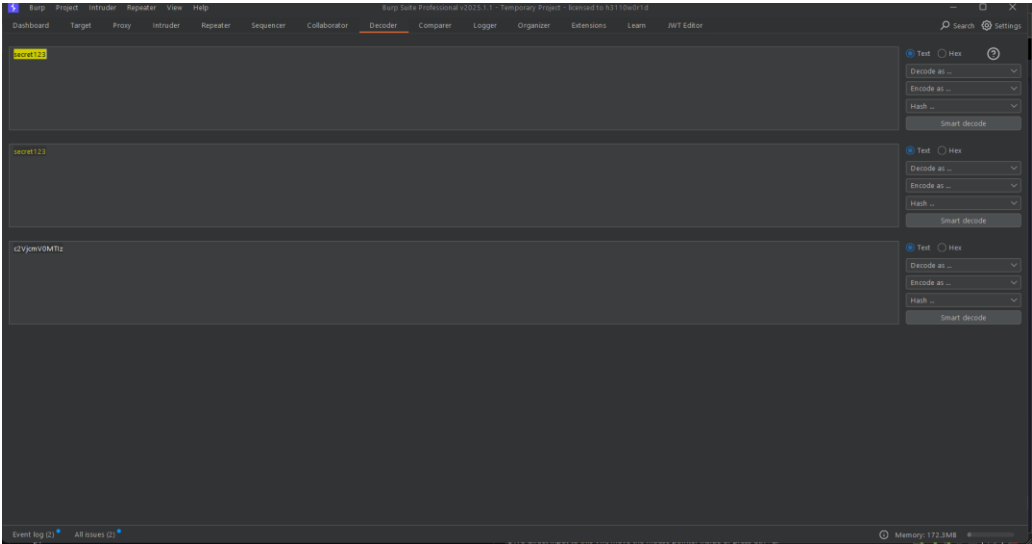
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQoiOiI2OTI5NDZmY2JkMzE2ZTlkM2QzYzQ3M2IiLCJ1c2VybmFtZSI6ImhhY2tldiIsIm1hZCI6ImR5c2NDQwMDA5MH0uPhdQOiI7Bi1zzjkHnaN4s0j041hWXXkCRPsauqVjTZXkk:Secret123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 16500 (JWT (JSON Web Token))
Hash.Target.....: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQoiOi... jTZXkk
Time.Started.....: Sat Nov 29 05:06:31 2025 (1 sec)
Time.Estimated...: Sat Nov 29 05:06:32 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 260.5 kH/s (1.83ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 17408/14344385 (0.12%)
Rejected.....: 0/17408 (0.00%)
Restore.Point....: 16384/14344385 (0.11%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: christal → petey
Hardware.Mon.#1..: Util: 37%

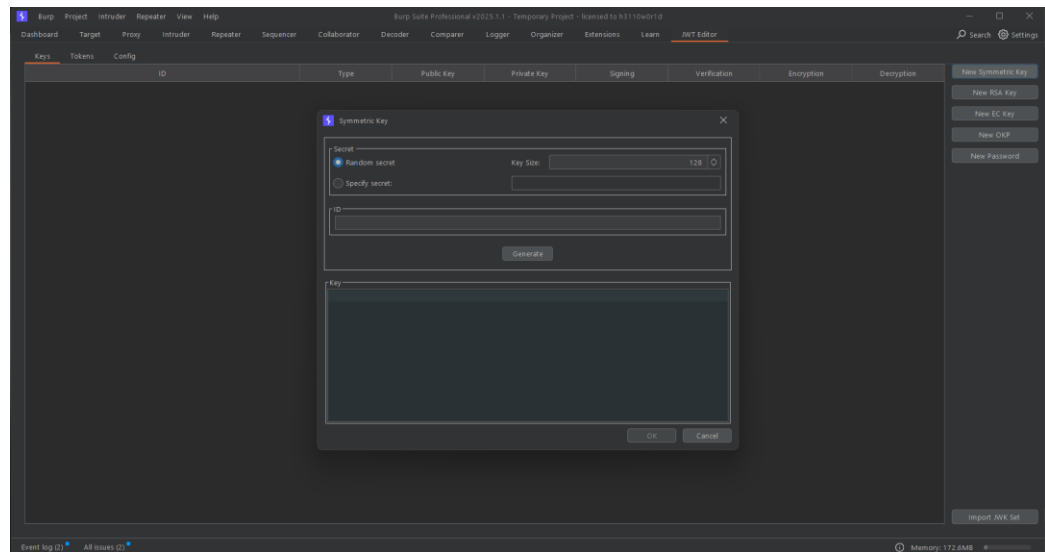
Started: Sat Nov 29 05:06:29 2025
Stopped: Sat Nov 29 05:06:33 2025

└─(hacker@suraj)-[~]
```

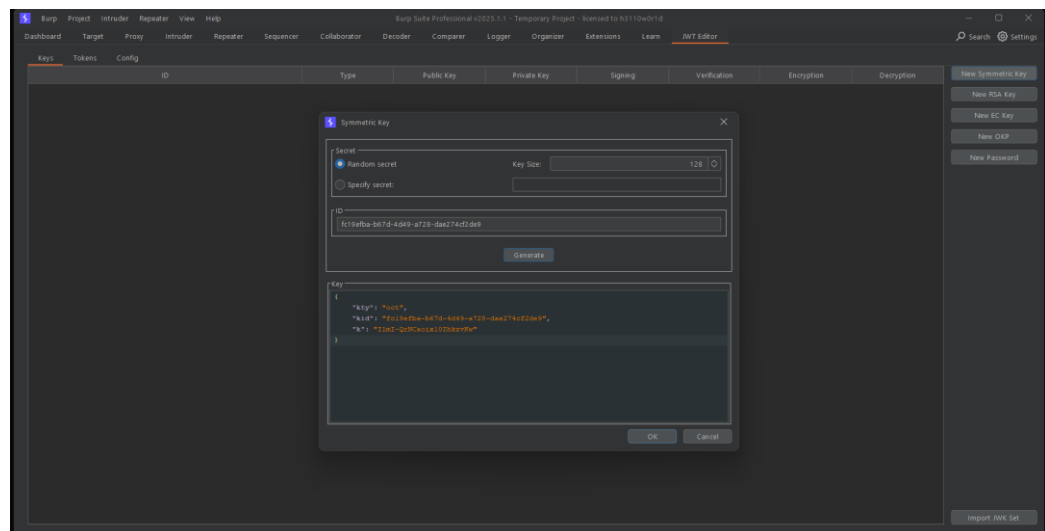
- Got secret: **secret123**
- Let's sign the token
- Go to burp Decoder tab and paste the secret key
- Encode as Base-64
- Copy the value



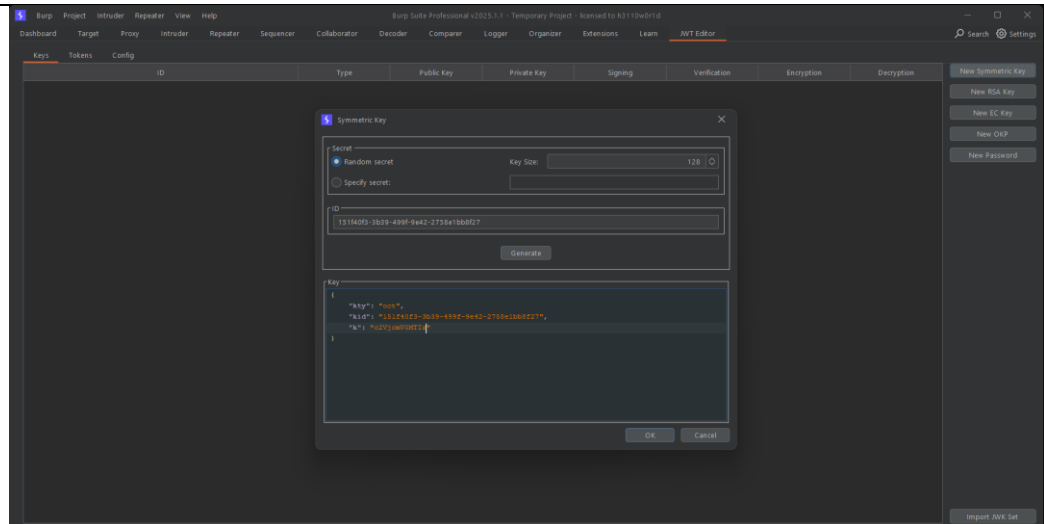
- Go to JWT Editor and click on **New symmetric key**



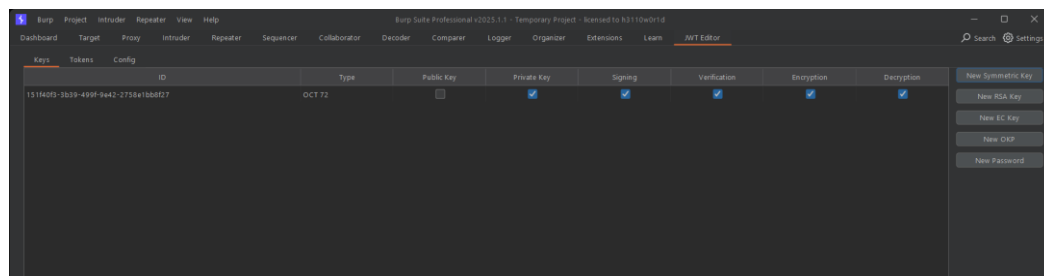
- Click on Generate



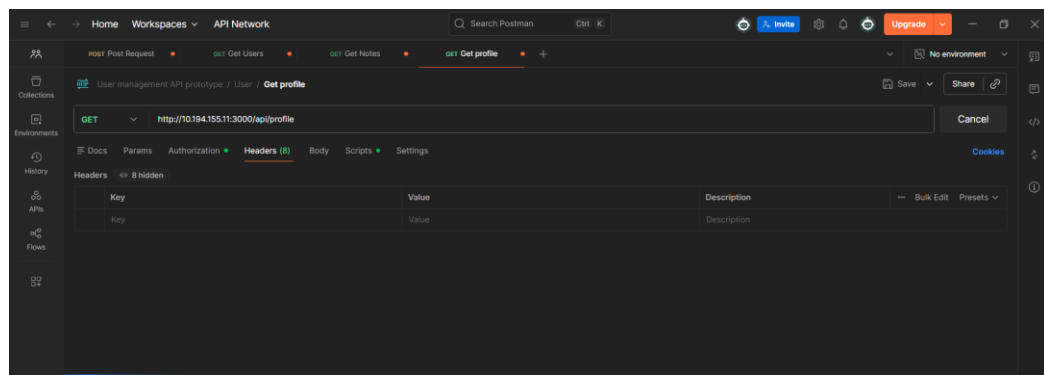
- Replace the value with the Secret that we have encoded in key field



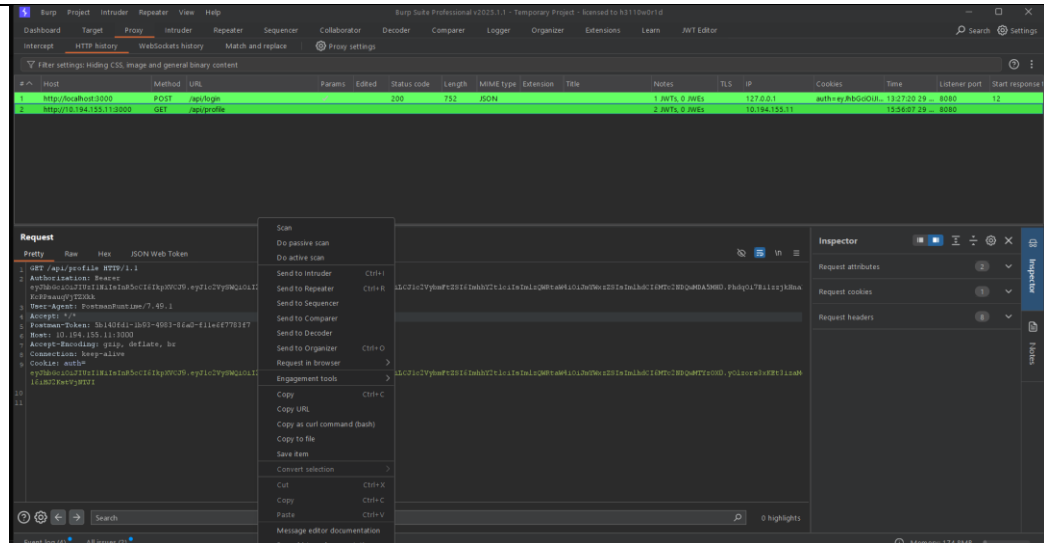
- Click OK. Now we can use it to sign any token



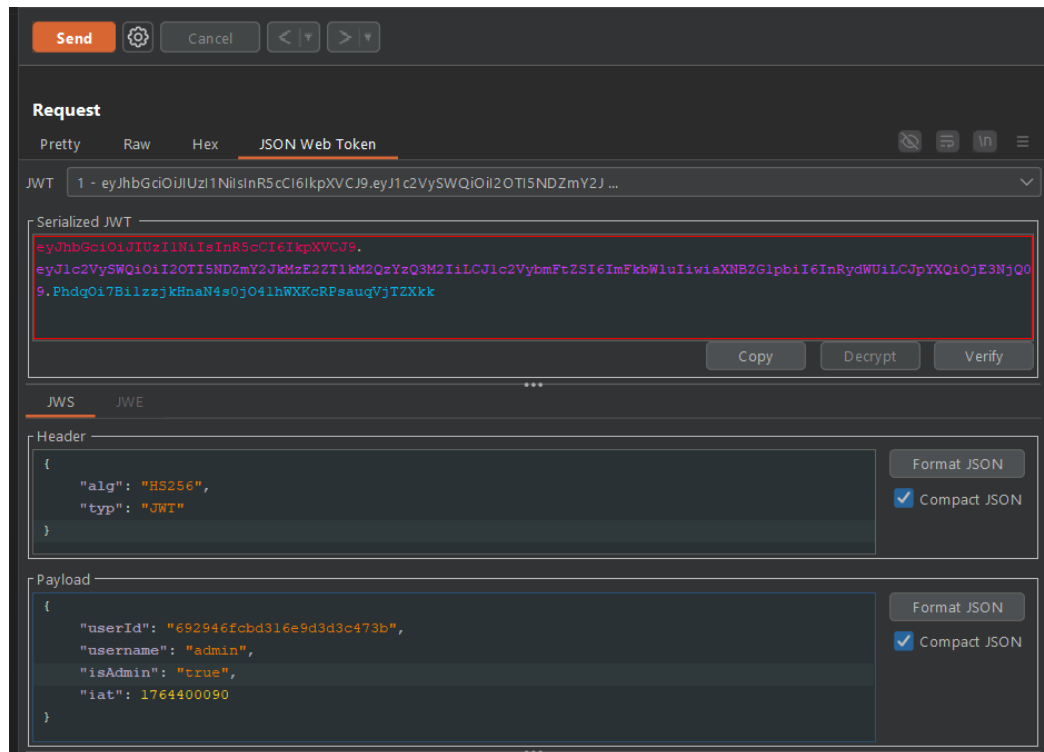
STEP 10 – From Postman try to access **/api/profile** endpoint and try to tamper token and access admin user profile



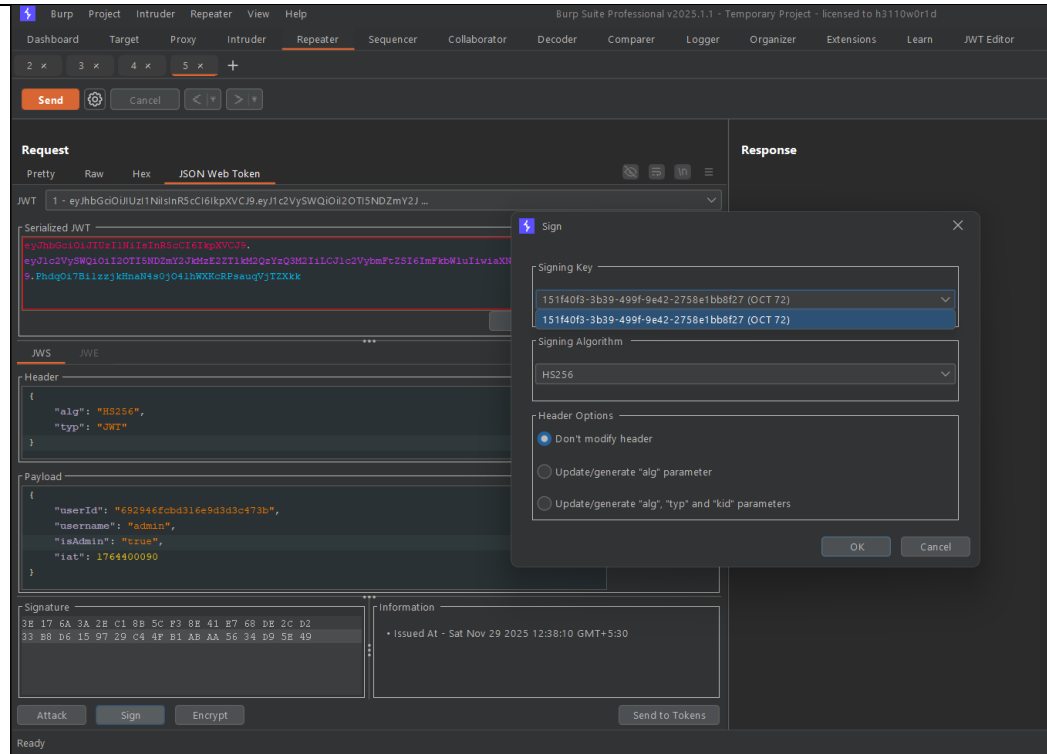
- Request got intercepted in Burp Suite



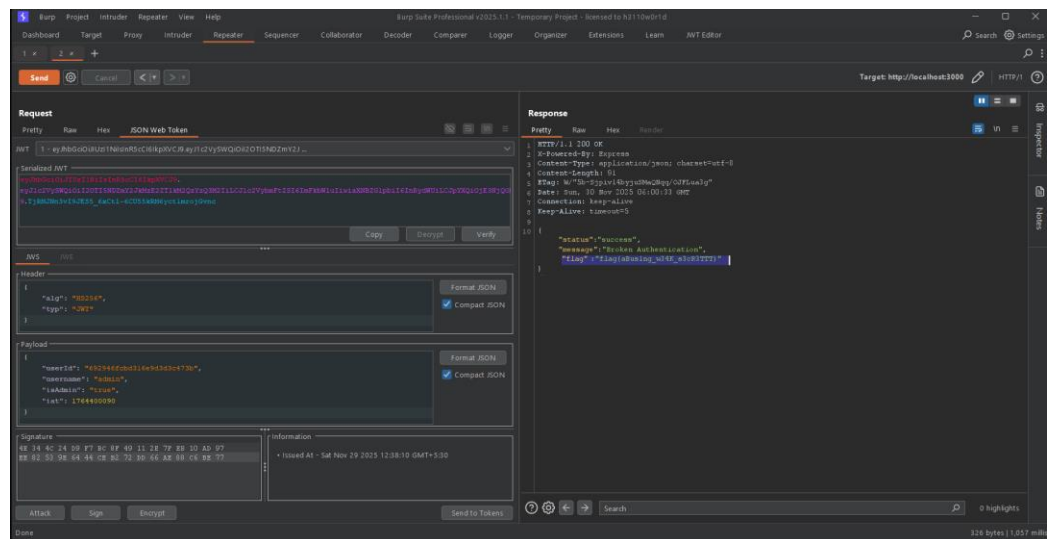
- Let's send to repeater
- Modify the token values



- Click on Sign & Select the secret we have created



- Then send the request



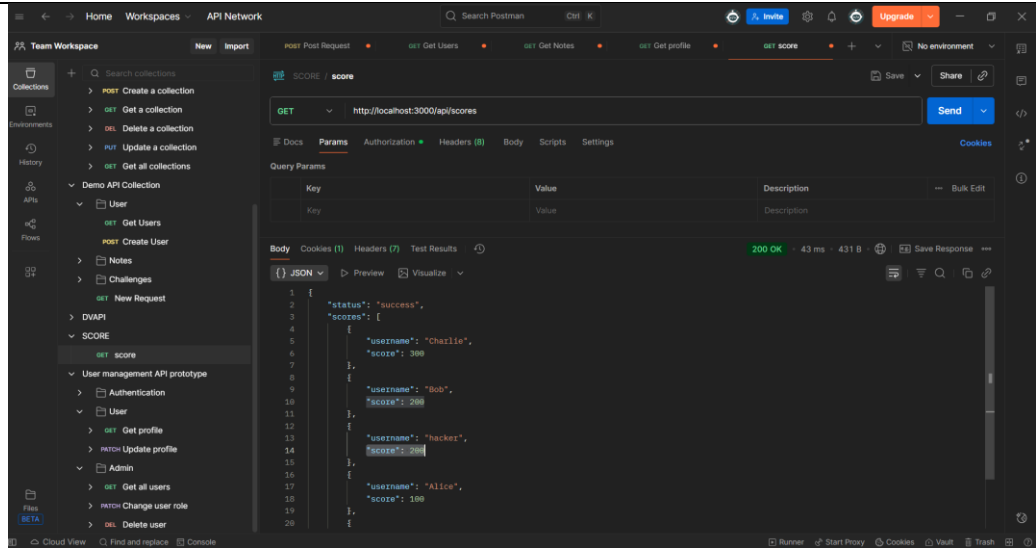
- Here we got Flag

Impact

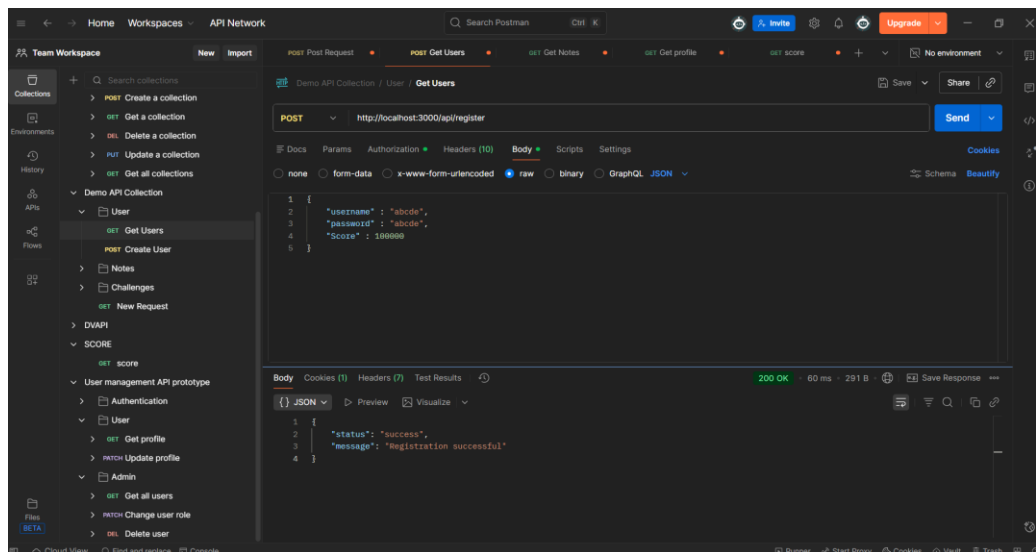
Attackers can gain complete control of other user's accounts in the system, read their personal data, and perform sensitive actions on their behalf. Systems are unlikely to be able to distinguish attacker's actions from legitimate user ones.

Remediation	• Make sure you know all the possible flows to authenticate to the API (mobile/ web / deep links that implement one click authentication / etc.). Ask your engineers what flows you missed. • Read about your authentication mechanisms. Make sure you understand what and how they are used. OAuth is not authentication, neither are API keys. • Don't reinvent the wheel in authentication, token generation, or password storage. Use the standards. • Credential recovery/forgot password endpoints should be treated as login endpoints in terms of brute force, rate limiting, and lockout protections. • Require re-authentication for sensitive operations (e.g., changing the account owner email address/2FA phone number). • Use the OWASP Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html • Implement anti-brute force mechanisms to mitigate credential stuffing, dictionary attacks, and brute force attacks on your authentication endpoints. This mechanism should be stricter than the regular rate limiting mechanisms on your APIs.
Reference	https://owasp.org/API-Security/editions/2023/en/0x11-api2-broken-authentication/

Vulnerability Name	Broken Object Property Level Authorization
Target URL	http://localhost:3000/api/scores
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – Here on endpoint, we can see the score

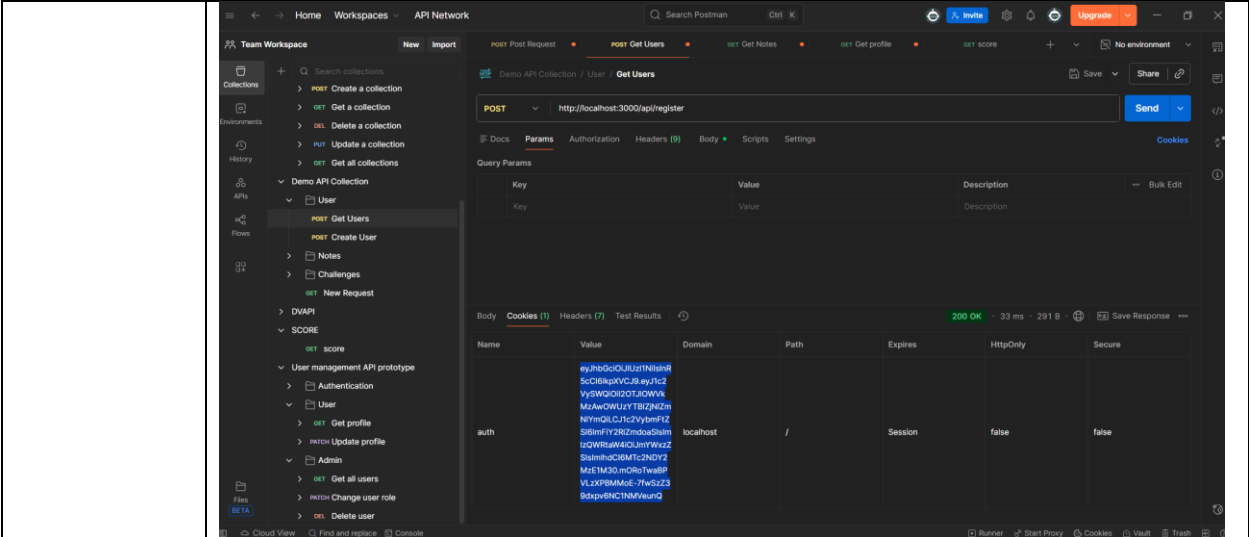


- So, there is key Value named Score as we can see in POC
- We can try Mass Assignment to Add-up parameters form client side
- Let's go for /register page and try to assign the score parameter for the new user we are registering

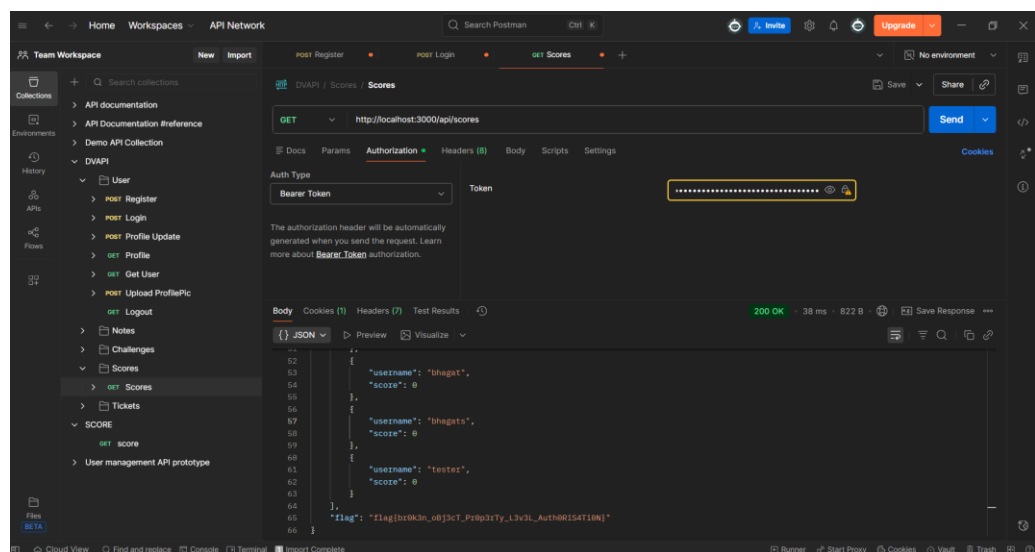


- Here we have passed the parameter we have seen that request is successful let's try to verify it get update on server side

WEB PENETRATION TESTING - DVAPI



- Let's try to copy and update the token and access to the try to fetch it on /api/scores
- Here we got the Misassignment means target is vulnerable

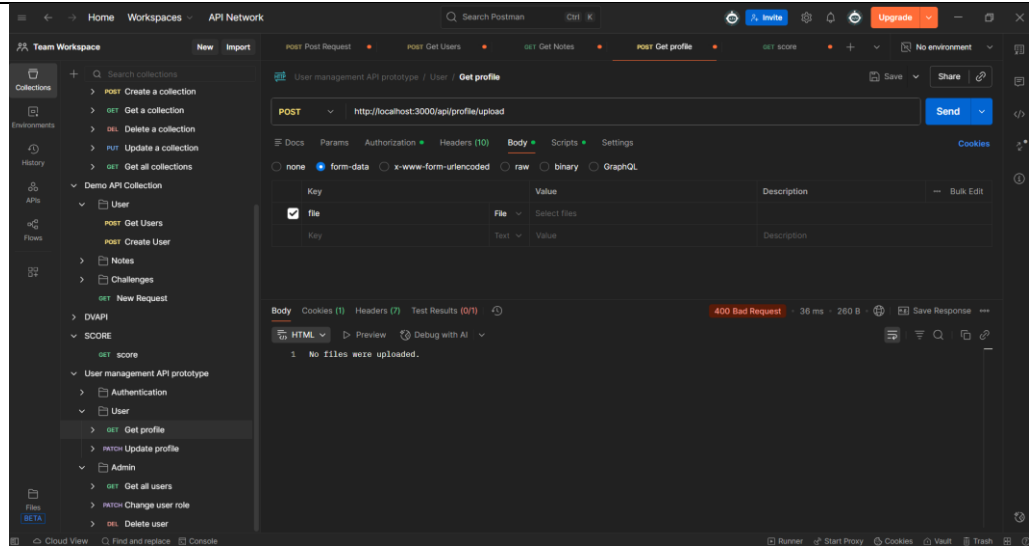


- We got the flag

Impact	Unauthorized access to private/sensitive object properties may result in data disclosure, data loss, or data corruption. Under certain circumstances, unauthorized access to object properties can lead to privilege escalation or partial/full account takeover
Remediations	When exposing an object using an API endpoint, always make sure that the user should have access to the object's properties you expose.

	• Avoid using generic methods such as <code>to_json()</code> and <code>to_string()</code>. Instead, cherry-pick specific object properties you specifically want to return. • If possible, avoid using functions that automatically bind a client's input into code variables, internal objects, or object properties ("Mass Assignment") • Allow changes only to the object's properties that should be updated by the client. • Implement a schema-based response validation mechanism as an extra layer of security. As part of this mechanism, define and enforce data returned by all API methods. • Keep returned data structures to the bare minimum, according to the business/functional requirements for the endpoint.
Reference	https://owasp.org/API-Security/editions/2023/en/Oxa3-broken-object-property-level-authorization/

Vulnerability Name	Unrestricted Resource Consumption
Target URL	http://localhost:3000/api/profile/upload
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – We have upload functionality on /upload API endpoint



API requests consume resources such as CPU/memory, bandwidth, storage, and integrations with other services. Attackers can cause high resource consumption by sending excessive requests, leading to unresponsive APIs or increased business expenses.

STEP 2 – Let's Download one image file and increase its size using dd command

```
(hacker@suraj)-[~/Desktop]
$ dd if=/dev/zero of=tiger.jpg bs=1M count=1000
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 35.0311 s, 29.9 MB/s

(hacker@suraj)-[~/Desktop]
$ ls
linux_payloads  solstice  tiger.jpg  Windows_payload

(hacker@suraj)-[~/Desktop]
$
```

- Let's try to share file on windows and try to upload on endpoint


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

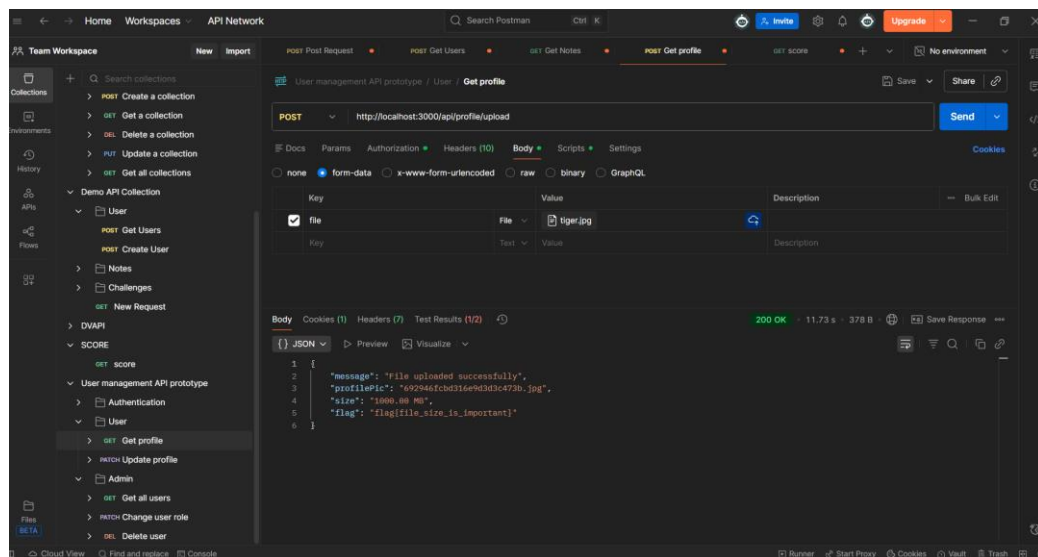
PS C:\Users\suraj> wget http://192.168.93.129:4444/tiger.jpg

StatusCode      : 200
StatusDescription : OK
Content         : {0, 0, 0, 0...}
RawContent      : HTTP/1.0 200 OK
                  Content-Length: 1048576000
                  Content-Type: image/jpeg
                  Date: Tue, 02 Dec 2025 08:54:23 GMT
                  Last-Modified: Tue, 02 Dec 2025 08:48:47 GMT
                  Server: SimpleHTTP/0.6 Python/3.12.7

Headers         : {[Content-Length, 1048576000], [Content-Type, image/jpeg], [Date,
                  Tue, 02 Dec 2025 08:54:23 GMT], [Last-Modified, Tue, 02 Dec 2025
                  08:48:47 GMT]...}
RawContentLength : 1048576000

PS C:\Users\suraj>
```

- Try to upload file tiger.jpg



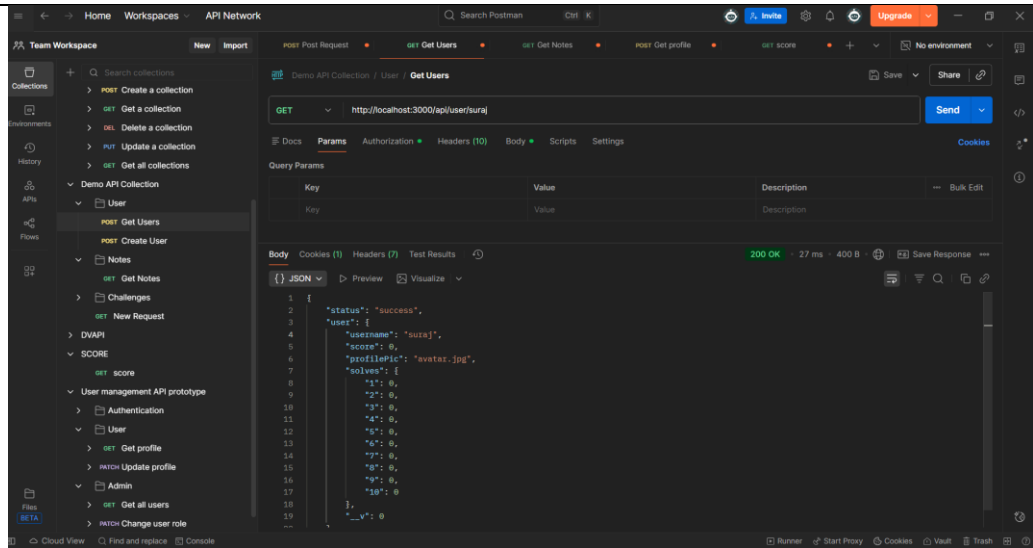
Here we uploaded file successfully and got the flag

Impact

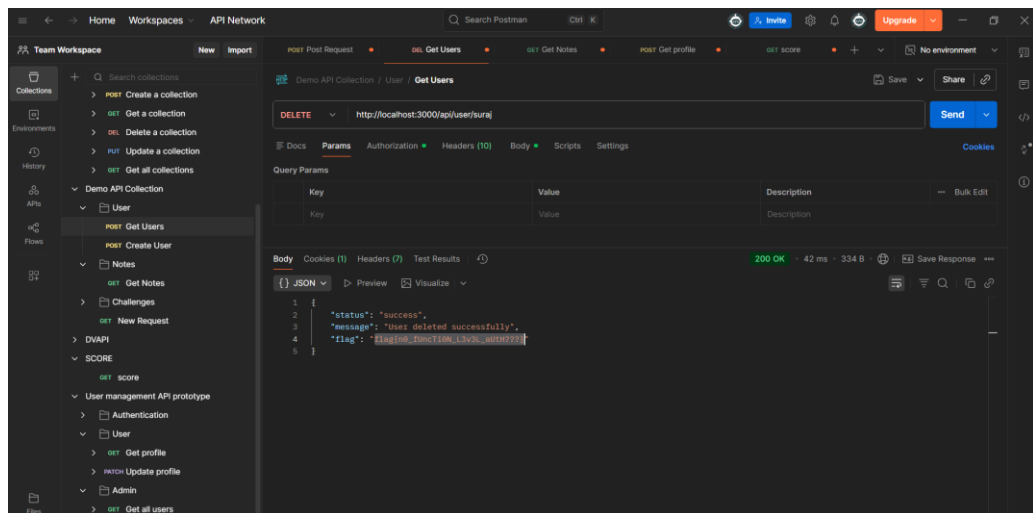
Exploitation can lead to DoS due to resource starvation, but it can also lead to operational costs increase such as those related to the infrastructure due to higher CPU demand, increasing cloud storage needs, etc.

Remediations	• Use a solution that makes it easy to limit memory, CPU, number of restarts, file descriptors, and processes such as Containers / Serverless code (e.g. Lambdas). • Define and enforce a maximum size of data on all incoming parameters and payloads, such as maximum length for strings, maximum number of elements in arrays, and maximum upload file size (regardless of whether it is stored locally or in cloud storage). • Implement a limit on how often a client can interact with the API within a defined timeframe (rate limiting). • Rate limiting should be fine tuned based on the business needs. Some API Endpoints might require stricter policies. • Limit/throttle how many times or how often a single API client/user can execute a single operation (e.g. validate an OTP, or request password recovery without visiting the one-time URL). • Add proper server-side validation for query string and request body parameters, specifically the one that controls the number of records to be returned in the response. • Configure spending limits for all service providers / API integrations. When setting spending limits is not possible, billing alerts should be configured instead.
Reference	https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/

Vulnerability Name	Broken Function Level Authorization
Target URL	http://localhost:3000/api/user/user
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – Try to see that if we can access another user's data through endpoint



STEP 2 – Let's try to DELETE user and we can access this functionality which is meant for admin only then its valid vulnerability in API



As we can see we have successfully deleted user and retrieved flag.

Impact

Such flaws allow attackers to access unauthorized functionality. Administrative functions are key targets for this type of attack and may lead to data disclosure, data loss, or data corruption. Ultimately, it may lead to service disruption

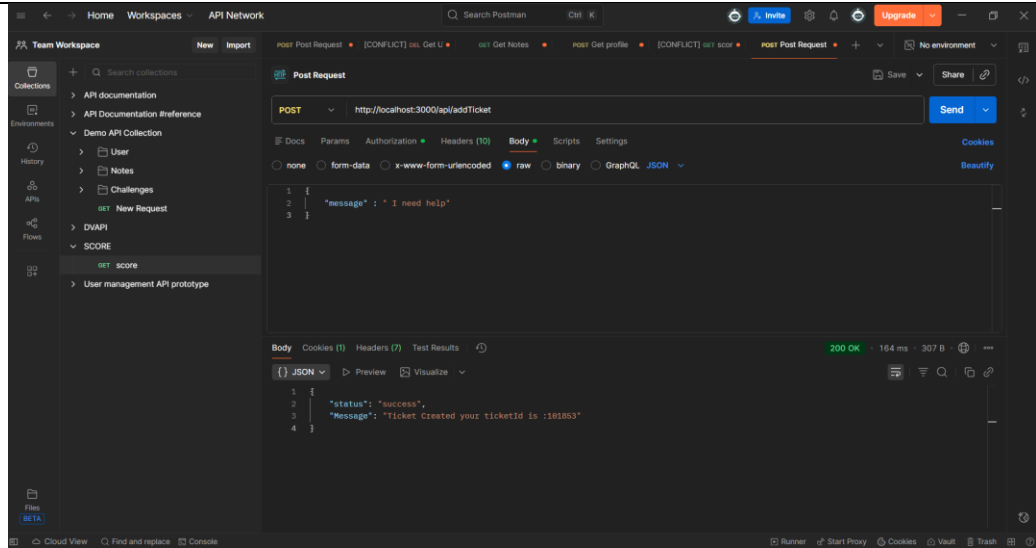
Remediations

This application should have a consistent and easy-to-analyze authorization module that is invoked from all their business functions. Frequently, such

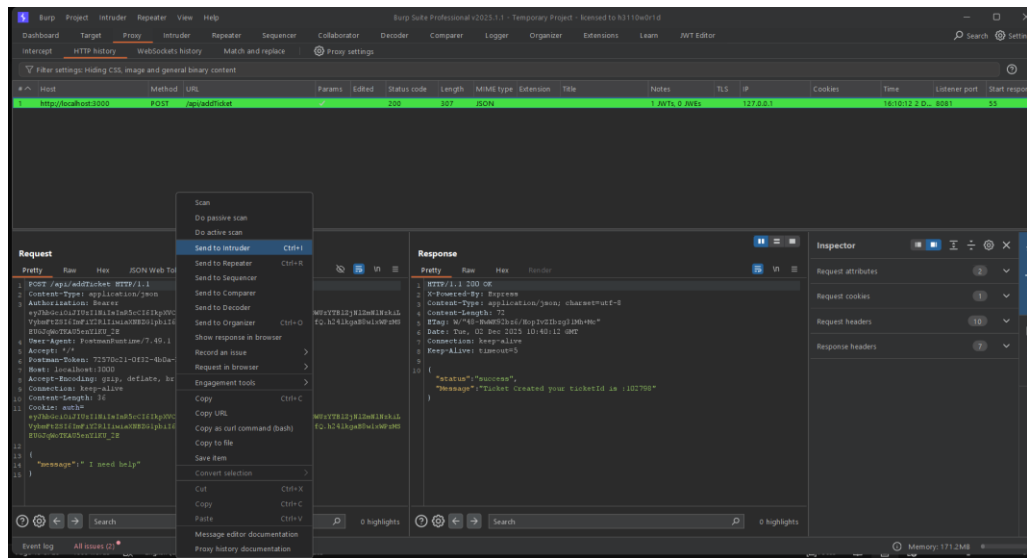
	protection is provided by one or more components external to the application code. • The enforcement mechanism(s) should deny all access by default, requiring explicit grants to specific roles for access to every function. • Review your API endpoints against function level authorization flaws, while keeping in mind the business logic of the application and groups hierarchy. • Make sure that all of your administrative controllers inherit from an administrative abstract controller that implements authorization checks based on the user's group/role. • Make sure that administrative functions inside a regular controller implement authorization checks based on the user's group and role.
Reference	https://owasp.org/API-Security/editions/2023/en/Oxa5-broken-function-level-authorization/

Vulnerability Name	Unrestricted Access to Sensitive Business Flows
Target URL	http://localhost:3000/api/getticket
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 - This endpoint allows users to create internal support tickets. During testing, it was found that this endpoint is accessible without authentication, indicating improper access control.

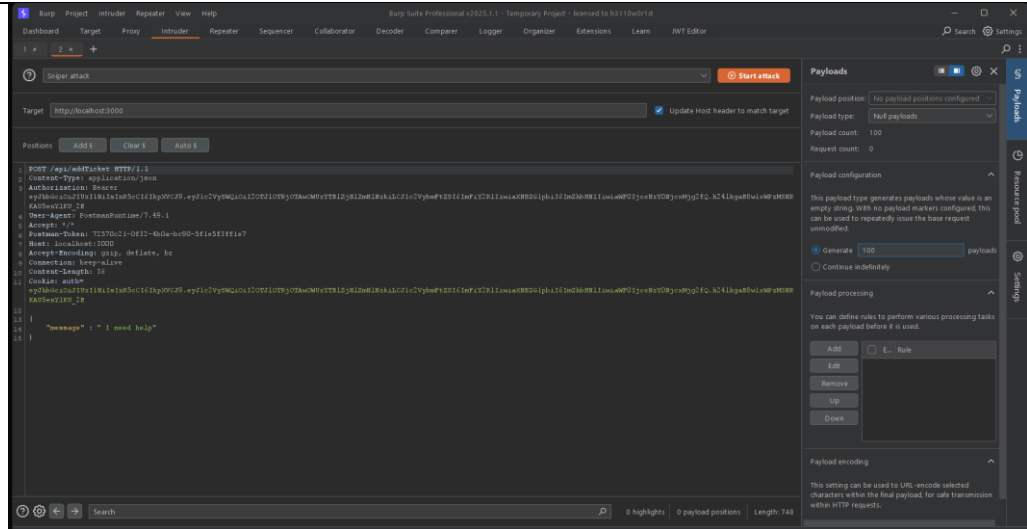
WEB PENETRATION TESTING - DVAPI



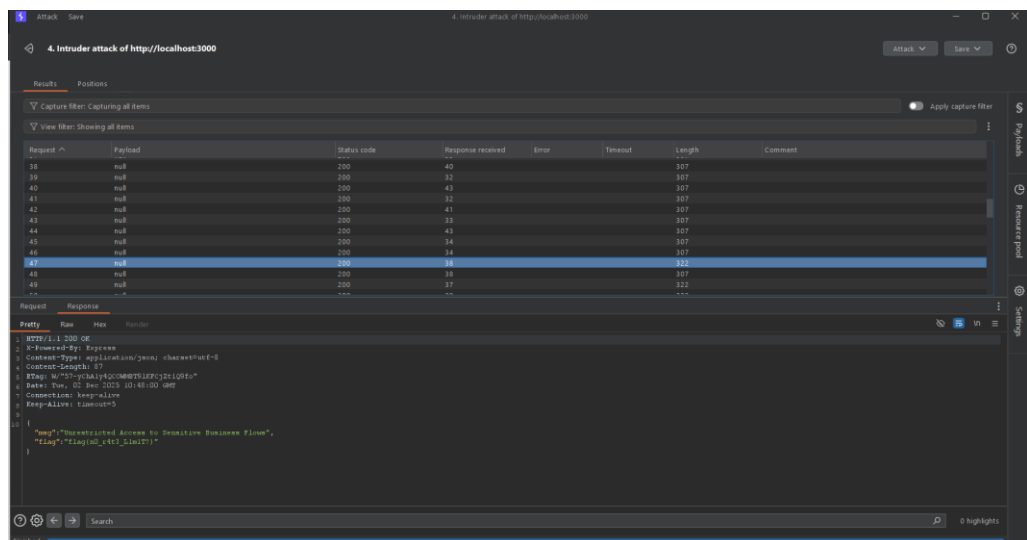
STEP 2 – Send request to burp to intercept and send it to Intruder



STEP 3 – In Intruder set Null payloads



STEP 4 – Start Attack and after filter using length

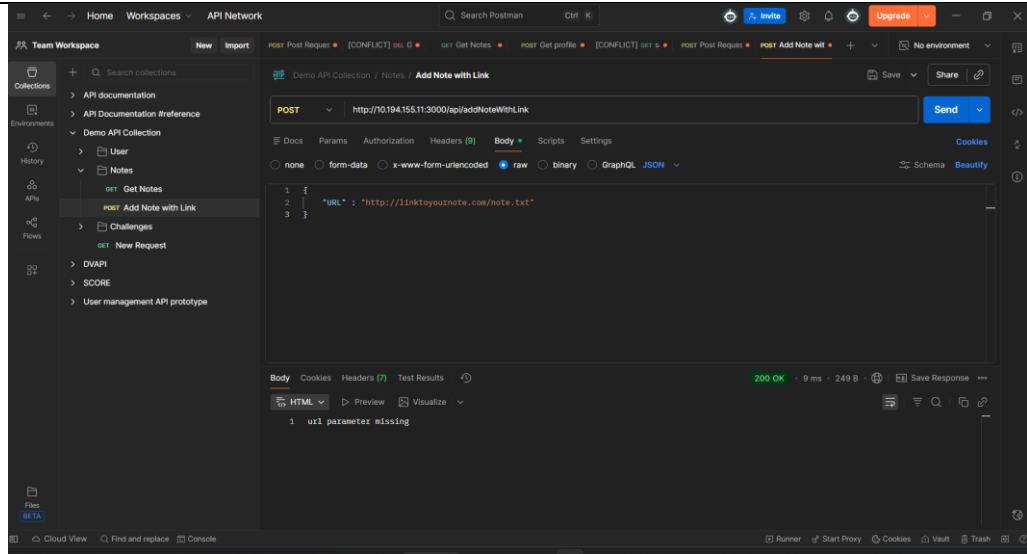


We got the flag

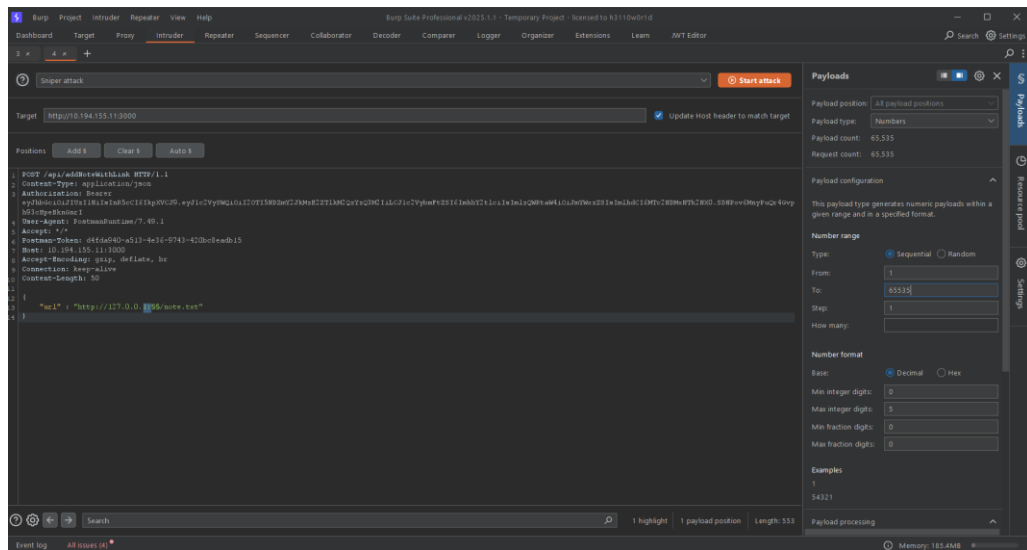
Impact	In general, technical impact is not expected. Exploitation might hurt the business in different ways, for example: prevent legitimate users from purchasing a product, or lead to inflation in the internal economy of a game.
Remediations	- Business – Identify the business flows that might harm the business if they are excessively used. - Engineering – Choose the right protection mechanisms to mitigate the business risk.

	Some of the protection mechanisms are simpler while others are more difficult to implement. The following methods are used to slow down automated threats: • Device fingerprinting: denying service to unexpected client devices (e.g. headless browsers) tends to make threat actors use more sophisticated solutions, thus more costly for them. • Human detection: using either captcha or more advanced biometric solutions (e.g. typing patterns) • Non-human patterns: analyze the user flow to detect non-human patterns (e.g. the user accessed the “add to cart” and “complete purchase” functions in less than one second) • Consider blocking IP addresses of Tor exit nodes and well-known proxies Secure and limit access to APIs that are consumed directly by machines (such as developer and B2B APIs). They tend to be an easy target for attackers because they often don’t implement all the required protection mechanisms.
Reference	https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/

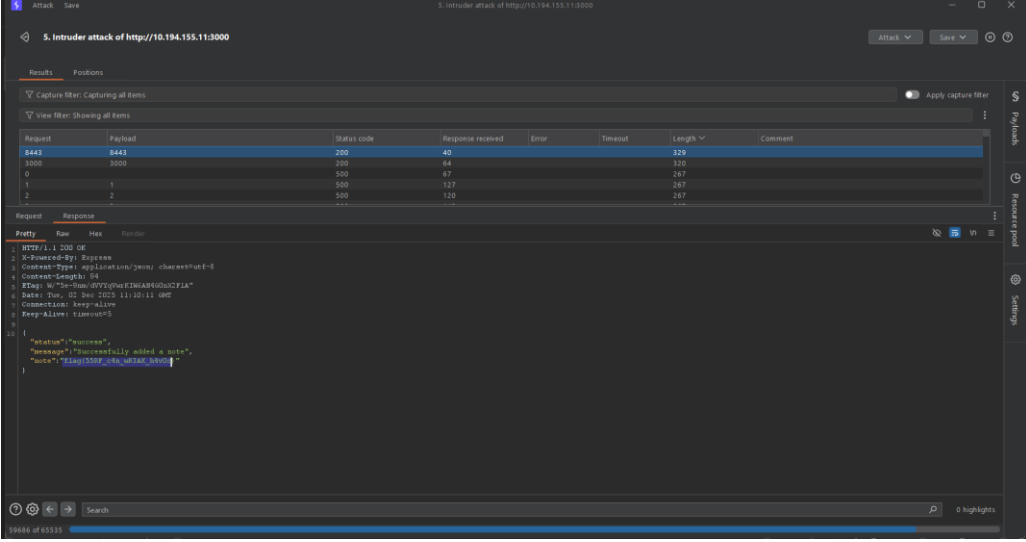
Vulnerability Name	Server-Side Request Forgery
Target URL	http://localhost:3000/api/addNoteWithLink
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – We have parameter it is requesting resource from a particular link



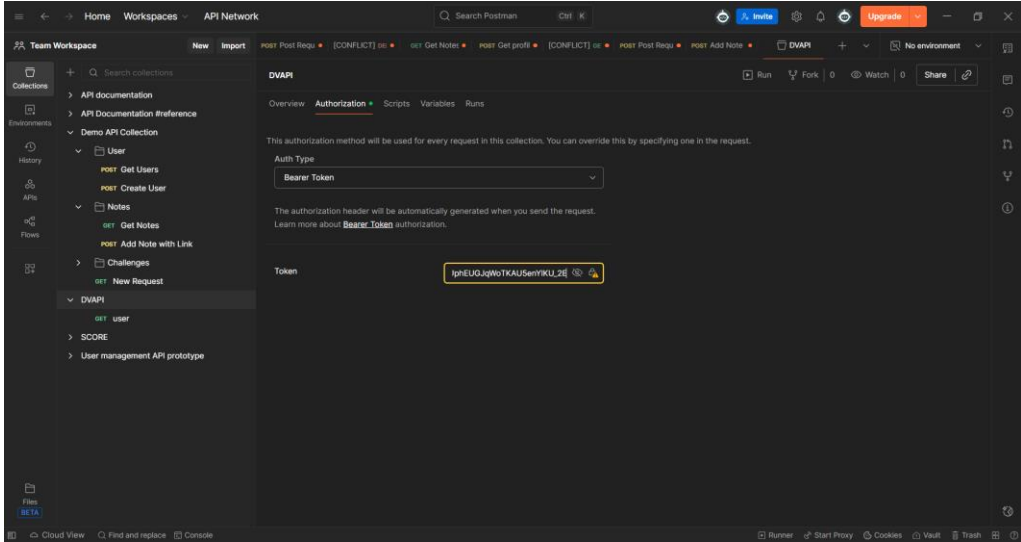
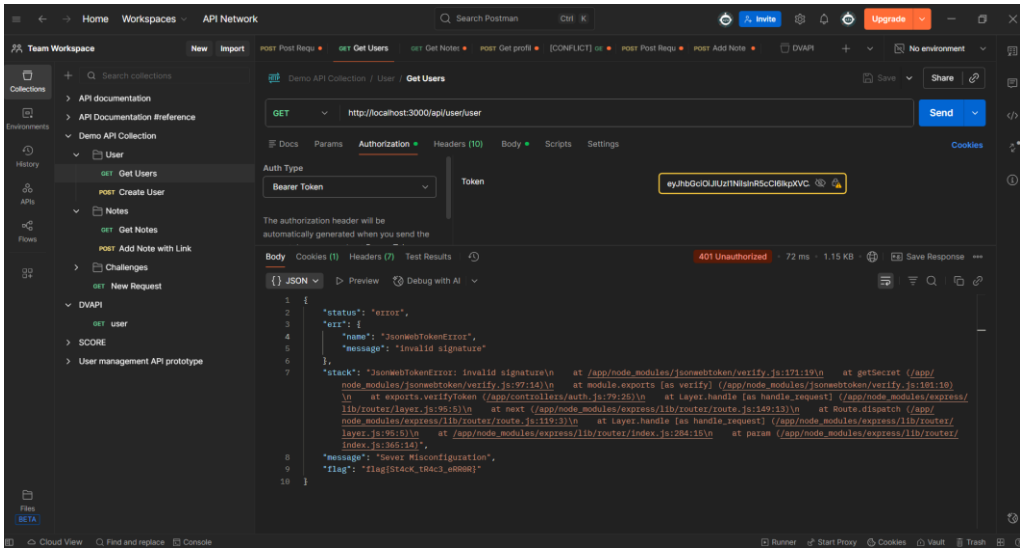
STEP 2 – Let's Modify it to local host and try to fetch from local host and fuzz port number with burp Intruder



STEP 3 – Start Attack and we got the flag

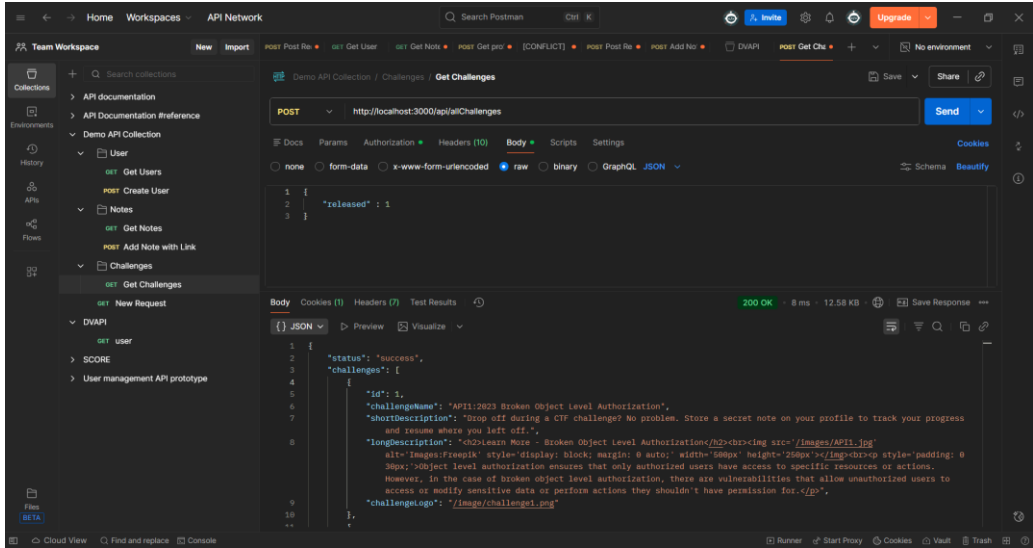
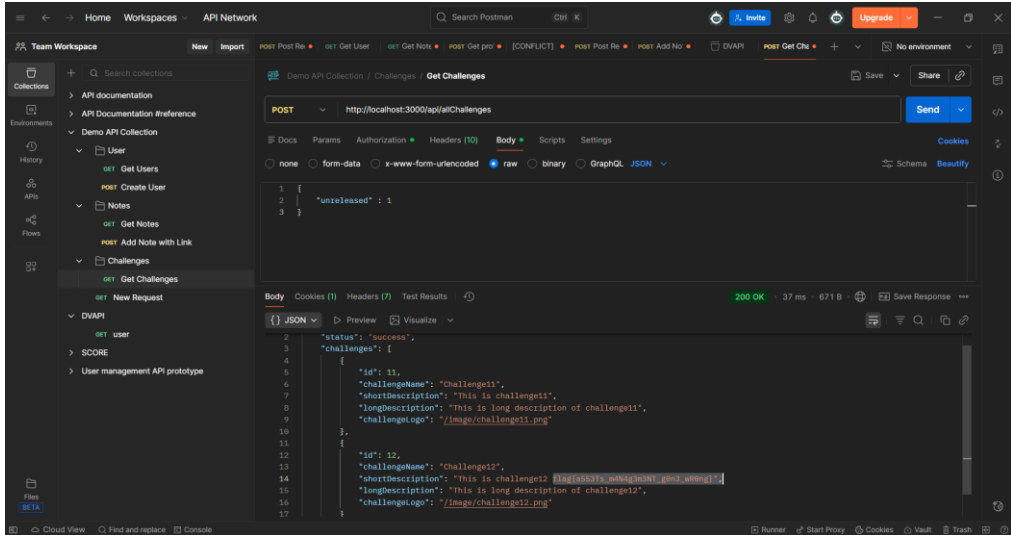
	 The screenshot shows the Burp Suite interface during an intruder attack. The top bar indicates the target is '5. Intruder attack of http://10.194.155.11:3000'. Below this, there's a table with columns: Request, Payload, Status code, Response received, Error, Timeout, Length, and Comment. The table shows several entries, with the first one having a status code of 200 and a response length of 40. Below the table, the 'Request' and 'Response' tabs are visible. The 'Request' tab shows the raw HTTP request, and the 'Response' tab shows the raw HTTP response. The response body contains a JSON object with a 'status' field set to 'success' and a 'message' field indicating a successful addition of a user.
Impact	Successful exploitation might lead to internal services enumeration (e.g. port scanning), information disclosure, bypassing firewalls, or other security mechanisms. In some cases, it can lead to DoS or the server being used as a proxy to hide malicious activities.
Remediations	- Isolate the resource fetching mechanism in our network: usually these features are aimed to retrieve resources and not internal ones. - Whenever possible, use allow lists of: - Remote origins users are expected to download resources from (e.g. Google Drive, Gravatar, etc.). - URL schemes and ports - Accepted media types for a given functionality - Disable HTTP redirections. - Use a well-rested and maintained URL parser to avoid issues caused by URL parsing inconsistencies. - Validate and sanitize all client-supplied input data. - Do not send raw Responses to clients.
Reference	https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/

Vulnerability Name	Security Misconfiguration
Target URL	http://localhost:3000/api/user/user

Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – Modify the token and try if we can fetch the details  STEP 2 – We removed last letter from token and saved and try to fetch profile  We got the flag

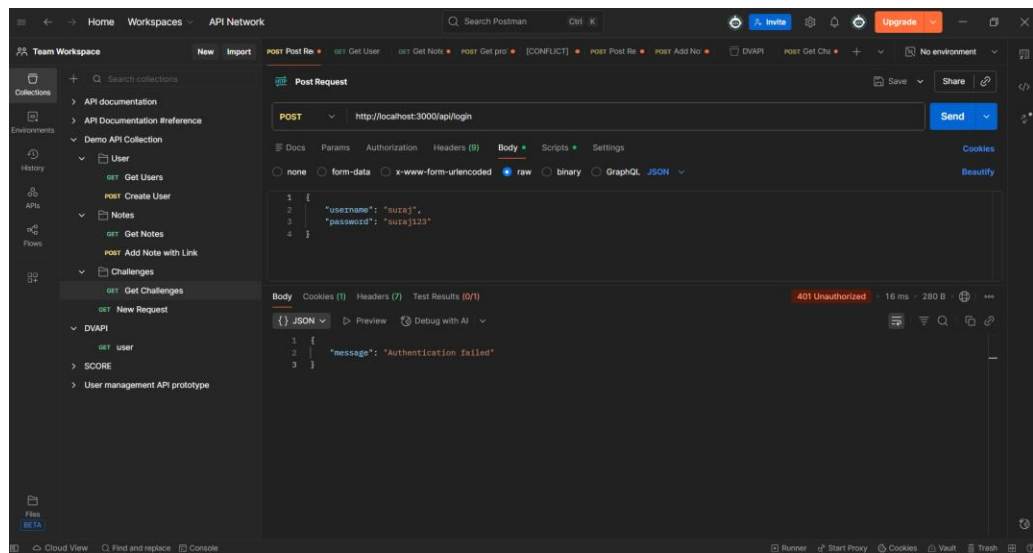
Impact	Security misconfigurations not only expose sensitive user data, but also system details that can lead to full server compromise
Remediations	The API life cycle should include: • A repeatable hardening process leading to fast and easy deployment of a properly locked down environment. • A task to review and update configurations across the entire API stack. The review should include: orchestration files, API components, and cloud services (e.g. S3 bucket permissions) • An automated process to continuously access the effectiveness of the configuration and settings in all environments Furthermore: • Ensure that all API communications from the client to the API server and any downstream/upstream components happen over an encrypted communication channel (TLS), regardless of whether it is an internal or public-facing API. • Be specific about which HTTP verbs each API can be accessed by: all other HTTP verbs should be disabled (e.g. HEAD). • APIs expecting to be accessed from browser-based clients (e.g. WebApp front-end) should, at least: • Implement a proper Cross-Origin Resource Sharing (CORS) policy • Include applicable Security Headers • Restrict incoming content types/data formats to those that meet the business/ functional requirements. • Ensure all servers in the HTTP server chain (e.g. load balancers, reverse and forward proxies, and back-end servers) process incoming requests in a uniform manner to avoid desync issues. • Where applicable, define and enforce all API response payload schemas, including error responses, to prevent exception traces and other valuable information from being sent back to attackers.
Reference	https://owasp.org/API-Security/editions/2023/en/Oxa8-security-misconfiguration/

Vulnerability Name	Improper Inventory Management
Target URL	http://localhost:3000/api/allChallenges
Severity	Critical

CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – Developer creates new endpoint try to exploit that fields or endpoint  STEP 2 – Change field to unreleased  We got the flag
Impact	Attackers can gain access to sensitive data, or even take over the server. Sometimes different API versions/deployments are connected to the same database with real data. Threat agents may exploit deprecated endpoints available in old API versions to get access to administrative functions or exploit known vulnerabilities.

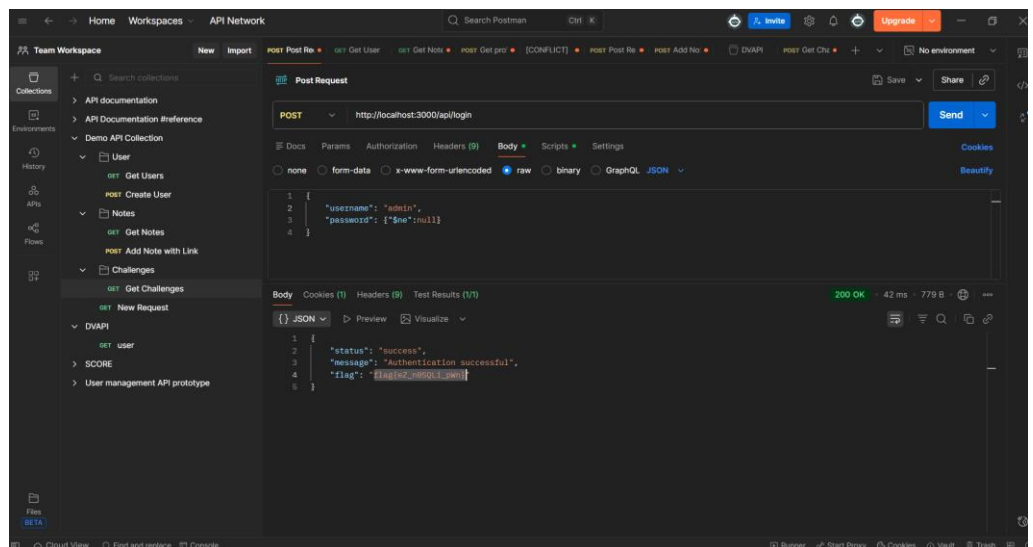
Remediations	• Inventory all API hosts and document important aspects of each one of them, focusing on the API environment (e.g. production, staging, test, development), who should have network access to the host (e.g. public, internal, partners) and the API version. • Inventory integrated services and document important aspects such as their role in the system, what data is exchanged (data flow), and their sensitivity. • Document all aspects of your API such as authentication, errors, redirects, rate limiting, cross-origin resource sharing (CORS) policy, and endpoints, including their parameters, requests, and responses. • Generate documentation automatically by adopting open standards. Include the documentation build in your CI/CD pipeline. • Make API documentation available only to those authorized to use the API. • Use external protection measures such as API security specific solutions for all exposed versions of your APIs, not just for the current production version. • Avoid using production data with non-production API deployments. If this is unavoidable, these endpoints should get the same security treatment as the production ones. • When newer versions of APIs include security improvements, perform a risk analysis to inform the mitigation actions required for the older versions. For example, whether it is possible to backport the improvements without breaking API compatibility or if you need to take the older version out quickly and force all clients to move to the latest version.
Reference	https://owasp.org/API-Security/editions/2023/en/Oxa9-improper-inventory-management/

Vulnerability Name	Unsafe Consumption of APIs
Target URL	http://localhost:3000/api/login
Severity	Critical
CVSS Score	9.8
Attack Vector	CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Proof Of Concept	STEP 1 – We have login endpoint lets try injection payloads to test is there any input validation



- We got authentication failed

STEP 2 – Let's try to inject NoSQL payload



We got the flag

Impact	The impact varies according to what the target API does with pulled data. Successful exploitation may lead to sensitive information exposure to unauthorized actors, many kinds of injections, or denial of service.
Remediations	- When evaluating service providers, assess their API security posture. - Ensure all API interactions happen over a secure communication channel (TLS).

	• Always validate and properly sanitize data received from integrated APIs before using it. • Maintain an allowlist of well-known locations integrated APIs may redirect yours to: do not blindly follow redirects
Reference	https://owasp.org/API-Security/editions/2023/en/Oxaa-unsafe-consumption-of-apis/